



ALBUKHARY INTERNATIONAL UNIVERSITY

CENTRE FOR FOUNDATION AND GENERAL STUDIES

SEMESTER 2 2025/2026

COURSE CODE : FAA 124
COURSE NAME : INTRODUCTION TO PROGRAMMING
EVALUATION : GROUP ASSIGNMENT
LECTURER'S NAME : SYAZA LYANA MAHADZIR
SUBMISSION DATE : 13/5/2026

No	NAME	STUDENT ID
1	SHUAIB AHAMAD	AIU25101084
2	KAUNG THIHA	AIU25101096
3	MD NABIDUL HUDA	AIU25101093
4	MOHAMED IRHAM	AIU25101072
5	AHMED ISMAIL ABDALLAH	AIU25101119

1. Program Overview

The Student Grade Management System is a console-based C program that allows users to manage student academic records at Albukhary International University. It supports adding students, recording grades across five subjects, computing averages, assigning letter grades, editing records, deleting students, and identifying the top performer.

1.1 Constants and Global Variables

Identifier	Description
MAX_STUDENTS 5	Maximum number of students that can be stored
MAX_SUBJECTS 5	Number of subjects per student (fixed)
NAME_LEN 50	Maximum character length for any name string
totalStudents	Global integer tracking how many students are currently stored
subjectNames[] []	Global array of 5 predefined subject name strings
students[]	Global array of Student structs (the main data store)

1.2 The Student Struct

All student data is held in a custom struct defined as:

```
typedef struct {
    char name[NAME_LEN];           // Student full name
    int id;                        // Auto-assigned unique ID
    float grades[MAX_SUBJECTS];    // One grade per subject
    float average;                 // Computed mean of all grades
    char letterGrade;              // A / B / C / D / F
} Student;
```

2. Function Reference

The program is divided into 10 functions. Each is described in detail below.

2.1 main()

Field	Details
Signature	int main(void)
Returns	0 on successful exit
Purpose	Entry point of the program. Displays the welcome banner, then runs a do-while loop that repeatedly shows the menu and dispatches user choices to the appropriate function.

How It Works

- Prints a one-time welcome banner and separator line.
- Enters a do-while loop that continues until the user selects option 7 (Exit).
- Calls printMenu() on every iteration to display the current options.
- Reads the user's integer choice with scanf().
- A switch-case block routes each choice:
 - Case 1 → addStudent()
 - Case 2 → viewAllStudents()
 - Case 3 → prompts for ID, then viewStudent(id)
 - Case 4 → prompts for ID, then editGrades(id)
 - Case 5 → prompts for ID, then deleteStudent(id)
 - Case 6 → displayTopStudent()
 - Case 7 → prints goodbye message, exits loop
 - default → prints invalid-choice error

2.2 printMenu()

Field	Details
Signature	void printMenu(void)
Returns	Nothing (void)
Purpose	Renders the numbered main menu to the console each time the loop iterates.

How It Works

- Prints separator lines and a MAIN MENU header.
- Lists all 7 menu options using printf().
- Purely cosmetic — no input reading or logic.

2.3 addStudent()

Field	Details
Signature	void addStudent(void)
Returns	Nothing (void)
Purpose	Collects a new student's name and grades, auto-assigns an ID, computes the average and letter grade, and stores the student in the global array.

How It Works

- Checks if totalStudents >= MAX_STUDENTS; if full, prints an error and returns early.
- Creates a temporary local Student struct s.
- Auto-assigns s.id = totalStudents + 26101000 (generates IDs starting at 26101000).
- Reads the student name using scanf("%[^\n]", s.name) to allow spaces in the name.
- Loops through all 5 subjects and reads a grade for each, then clamps the value to the 0–100 range:

```
if (grade < 0)    grade = 0;
if (grade > 100) grade = 100;
```

- Calls computeAverage() to calculate the mean across all 5 grades.
- Calls getLetterGrade() to assign a letter (A–F) based on the average.
- Copies the temporary struct into students[totalStudents] and increments totalStudents.
- Prints a success confirmation with the name, ID, average, and letter grade.

2.4 viewAllStudents()

Field	Details
Signature	void viewAllStudents(void)
Returns	Nothing (void)
Purpose	Prints a formatted table listing every currently stored student.

How It Works

- If `totalStudents == 0`, prints "No students found" and returns.
- Prints a table header row with columns: ID, Name, Average, Grade.
- Loops through the `students[]` array and calls `printStudentRow()` for each entry.
- After the loop, prints a footer showing the total student count.

2.5 printStudentRow()

Field	Details
Signature	<code>void printStudentRow(Student s)</code>
Parameter	<code>s</code> — a <code>Student</code> struct passed by value
Returns	Nothing (void)
Purpose	Prints a single row of the student table using consistent column widths.

How It Works

- Uses `printf()` with fixed-width format specifiers:

```
printf(" %-6d %-20s %-8.2f %-6c\n", s.id, s.name, s.average, s.letterGrade);
```

- `%-6d` → left-aligned integer in 6 characters for the ID.
- `%-20s` → left-aligned string in 20 characters for the name.
- `%-8.2f` → left-aligned float with 2 decimal places in 8 characters for the average.
- `%-6c` → left-aligned character in 6 characters for the letter grade.
- Receives the struct by value, so it does not modify the original.

2.6 viewStudent()

Field	Details
Signature	<code>void viewStudent(int id)</code>
Parameter	<code>id</code> — the student ID to look up
Returns	Nothing (void)

Purpose	Displays the full detail view for a single student, including individual subject grades.
---------	--

How It Works

- Calls `findStudent(id)` to get the array index of the matching student.
- If `findStudent()` returns -1, prints a not-found error and returns.
- Copies the found struct into a local variable `s` for safe access.
- Prints the student ID, name, and a formatted subject-grade table.
- Loops through all `MAX_SUBJECTS` grades and prints each with its subject name.
- Prints the average and letter grade at the bottom.
- This function is also called by `displayTopStudent()` to reuse the same formatted output.

2.7 editGrades()

Field	Details
Signature	<code>void editGrades(int id)</code>
Parameter	<code>id</code> — the student ID whose grades will be edited
Returns	Nothing (void)
Purpose	Allows the user to overwrite all grades for an existing student and recomputes the average and letter grade.

How It Works

- Calls `findStudent(id)`; returns early with an error if the student is not found.
- Prints the student name as a heading for context.
- Loops through all 5 subjects. For each subject it:
 - Shows the subject name and the current grade value.
 - Reads the new grade from the user.
 - Clamps the input to the valid 0–100 range.
 - Writes the new value directly into `students[idx].grades[i]`.
- After the loop, calls `computeAverage()` and `getLetterGrade()` to update `students[idx].average` and `students[idx].letterGrade`.
- Prints a confirmation showing the new average and letter grade.

2.8 deleteStudent()

Field	Details
Signature	void deleteStudent(int id)
Parameter	id — the student ID to remove
Returns	Nothing (void)
Purpose	Removes a student from the array by shifting all subsequent entries one position left, then decrements the student count.

How It Works

- Calls findStudent(id); returns early with an error if not found.
- Prints the name and ID of the student being deleted as confirmation.
- Runs a shift loop to fill the gap created by the deletion:

```
for (i = idx; i < totalStudents - 1; i++)
    students[i] = students[i + 1];
```
- Decrements totalStudents by 1.
- The last element in the old range is now logically unreachable because totalStudents no longer includes it.
- Prints a success confirmation message.

2.9 displayTopStudent()

Field	Details
Signature	void displayTopStudent(void)
Returns	Nothing (void)
Purpose	Finds the student with the highest average grade and displays their full detail view.

How It Works

- If totalStudents == 0, prints a no-data message and returns.
- Initialises topIdx = 0 and topAvg = -1 (ensures even a 0 average is captured).
- Loops through all students and compares each average to topAvg:

```
if (students[i].average > topAvg) { topAvg = students[i].average; topIdx = i; }
```

- After the loop, calls `viewStudent(students[topIdx].id)` to display the top student's full record.
- In the case of a tie, the student who appears first in the array (added earliest) is selected.

2.10 computeAverage()

Field	Details
Signature	<code>float computeAverage(float grades[], int count)</code>
Parameters	<code>grades[]</code> — array of grade values; <code>count</code> — number of elements to average
Returns	<code>float</code> — the computed arithmetic mean, or 0.0 if count is 0
Purpose	Pure helper function that computes the arithmetic mean of an array of floats.

How It Works

- Declares a local `float sum = 0.0f` and an index variable `i`.
- Iterates through `count` elements and accumulates the total into `sum`.
- Returns `sum / count` if `count > 0`, otherwise returns 0.0f to prevent division-by-zero.
- Called by both `addStudent()` and `editGrades()` after grade data is collected or updated.

2.11 getLetterGrade()

Field	Details
Signature	<code>char getLetterGrade(float average)</code>
Parameter	<code>average</code> — the numeric average to evaluate
Returns	<code>char</code> — a single letter: A, B, C, D, or F
Purpose	Maps a numeric average to a letter grade using a fixed grading scale.

Grading Scale

Average Range	Letter Grade
---------------	--------------

90.00 – 100.00	A
80.00 – 89.99	B
70.00 – 79.99	C
60.00 – 69.99	D
0.00 – 59.99	F

How It Works

- Uses a chain of if / else if / else comparisons from highest to lowest threshold.
- Returns the corresponding char literal the moment a match is found.
- Called by addStudent() and editGrades() immediately after computeAverage().

2.12 findStudent()

Field	Details
Signature	int findStudent(int id)
Parameter	id — the student ID to search for
Returns	int — the array index of the matching student, or -1 if not found
Purpose	Linear search helper that locates a student in the global array by their ID.

How It Works

- Iterates through students[0] to students[totalStudents - 1].
- Compares students[i].id against the target id.
- Returns i immediately on the first match (early exit).
- Returns -1 if no match is found after checking all entries.
- Used by viewStudent(), editGrades(), and deleteStudent() as a lookup gateway.

3. Program Execution Flow

The diagram below summarises how the functions connect at runtime:

Step	What Happens
1. Start	main() prints the welcome banner
2. Loop begins	printMenu() displays the 7 options
3. User input	scanf reads the integer choice
4. Dispatch	switch-case calls the relevant function
5. Data ops	Functions call findStudent(), computeAverage(), getLetterGrade() as needed
6. Output	Results printed; control returns to main()
7. Repeat	Loop continues until choice == 7
8. Exit	main() prints goodbye and returns 0

4. Known Limitations

The system stores data only in memory; all records are lost when the program exits (no file I/O).

- The maximum number of students is hard coded to 5 (MAX_STUDENTS). Changing it requires recompiling.
- Student IDs are sequential and are not re-used after deletion. Deleting and re-adding students may create ID gaps.
- Input validation is minimal: the program does not handle non-numeric input for the grade or choice fields (scanf will fail silently or loop).
- The grading scale is fixed at five grades (A–F) with no support for +/- modifiers.